Data Preprocessing with Pandas

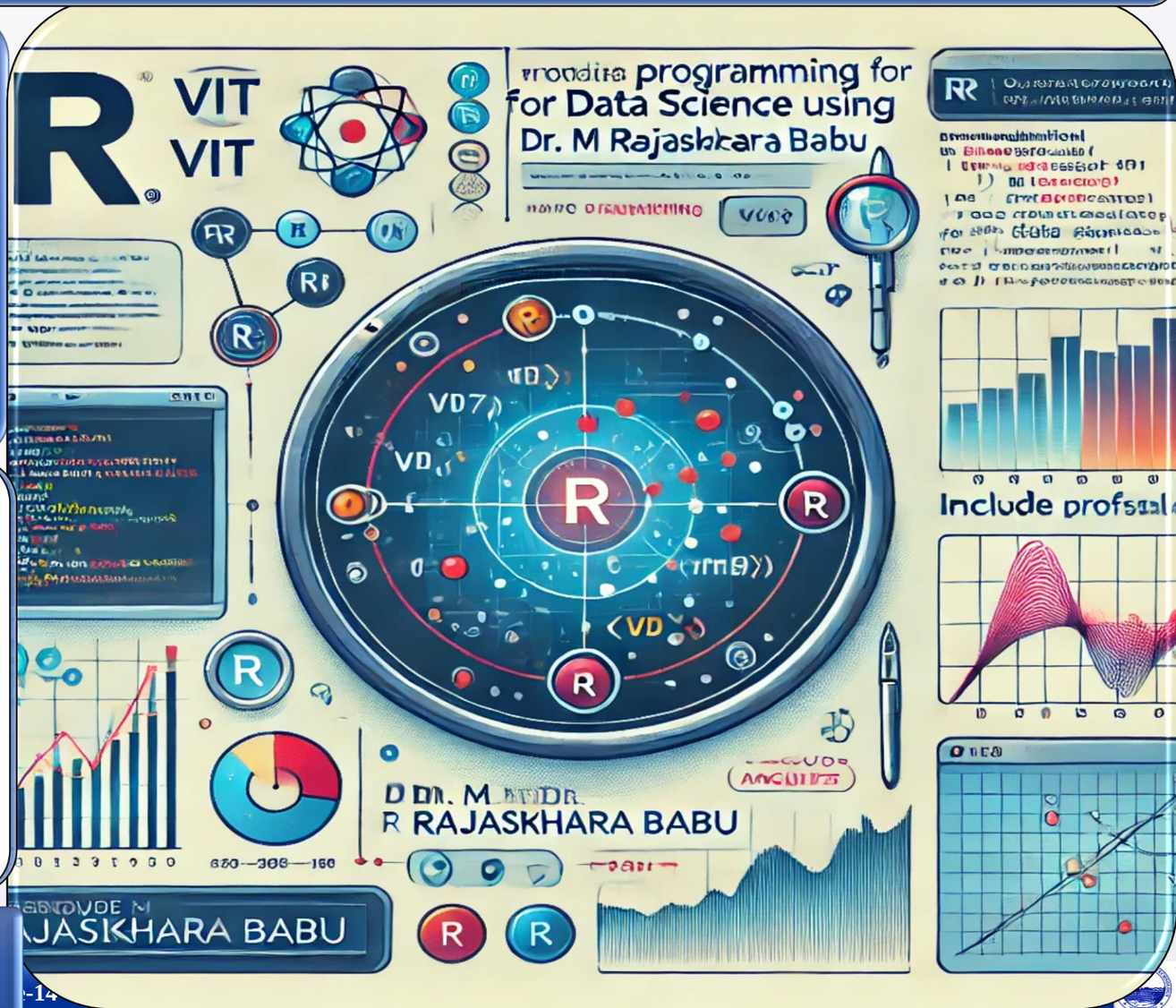
by

Dr M Rajasekhara Babu

School of

Computer Science and Engineering

https://www.rajababu.org



VELLORE INSTITUTE OF TECHNOLOGY
VIT
உழைப்பே உயர்வு தரும்

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India

Objectives & Teaching Learning Material

To explain the concepts and significance of data preprocessing in preparing datasets for analysis and machine learning

To demonstrate techniques for identifying, handling, and removing missing values using Pandas functions

To illustrate data transformation methods such as type conversion and Min-Max normalization for improving data quality and consistency

Teaching + Learning Material: LCD, White board Marker, Presentation slides

Learning outcomes

Upon successful completion of this lecture, students will be able to:

Identify and handle missing values in datasets using appropriate Pandas preprocessing techniques

Apply data transformation methods such as data type conversion and Min-Max normalization to improve data quality

Develop and execute a complete data preprocessing workflow for preparing data for analysis and machine learning applications

Session Plan

Time in minutes	Topic	Learning Aid & Methodology	Faculty Approach	Student Activity	Skill and Competency Developed
05	Re-Cap	Quiz Presentation	Questions Organizes	Answers Identifies	Knowledge Understanding
10	Introduction to Data Preprocessing and Missing Values	Presentation	Presentation	Listens	Remembering
12	Missing Value Detection and Handling	Explains	Explains	Notes	Understanding
13	Data Type Conversion and Normalization	Case Study	Organizes	Observes	Applying
10	Complete Preprocessing Pipeline and Applications	Discussion	Facilitates	Answers	Analyzing

Problem Statement

Data Cleaning and Preprocessing with Pandas: Create a DataFrame with some missing values in columns Name, Age, and City. Perform the following operations: fill missing values in the Age column with the mean age, drop rows where Name or City is missing, and convert the Age column to integer type. Finally, normalize the Age column using min-max scaling.

Problem Statement

We will work through a complete preprocessing pipeline on a small DataFrame with three columns: **Name**, **Age**, and **City** — containing intentional missing values.

1	2
Fill Missing Age Use the mean age to impute missing values in the Age column.	**Remove Incomplete Rows** Drop rows where Name or City is missing.
3	**4**
Convert Age to Integer Cast the Age column from float to int type.	**Normalize Age** Apply Min-Max Scaling to bring Age values into the 0–1 range.

Creating the Dataset

We begin by importing the required libraries and constructing a DataFrame with deliberate missing values (`None` and `np.nan`).

```
import pandas as pd
import numpy as np

data = {
} 'Name': ['Alice', 'Bob', None, 'David', 'Emma'],
    'Age':  [23, np.nan, 28, 35, np.nan],
    'City': ['New York', 'London', 'Paris', None, 'Sydney']

df = pd.DataFrame(data)
print(df)
```

Output:

Name	Age	City
Alice	23.0	New York
Bob	NaN	London
None	28.0	Paris
David	35.0	None
Emma	NaN	Sydney

Understanding Missing Values

Missing values represent unavailable or unknown data in a dataset. They appear in different forms depending on the source and library used.

NaN

Not a Number — the standard float representation of missing data in NumPy and Pandas.

None

Python's built-in null object, often used for missing string or object values.

Null

A general term for absent values, common in databases and other data systems.

Identifying Missing Values with `isnull()`

Before cleaning, we must locate where missing values exist. The `isnull()` method returns a boolean DataFrame — `True` where data is missing.

```
print(df.isnull())
```

Name	Age	City
False	False	False
False	True	False
True	False	False
False	False	True
False	True	False

- This step helps locate missing data before any cleaning begins — always inspect before acting.

Handling Missing Values

Why It Matters

Missing data can reduce accuracy, cause runtime errors, and significantly degrade machine learning model performance if left unaddressed.

Common Strategies

→ **Fill with Mean**

Best for numerical columns with roughly symmetric distributions.

→ **Fill with Median**

Preferred when data is skewed or has outliers.

→ **Fill with Mode**

Suitable for categorical columns.

→ **Remove Records**

Drop rows when missing data is too severe to impute reliably.

Calculating Mean Age

We use the mean of existing valid ages to fill in the missing values. Only non-null values are included in the calculation.

Existing Ages

The valid age values in our dataset are: **23, 28, 35**

Formula

$$\text{Mean} = \frac{\sum \text{Age}}{\text{Number of Valid Ages}}$$

Calculation

$$\text{Mean} = \frac{23 + 28 + 35}{3} = 28.67$$

The computed mean age of **28.67** will be used to replace all `NaN` values in the Age column.

Filling Missing Values with `fillna()`

We calculate the mean and apply it to replace all `NaN` entries in the Age column using `fillna()`.

```
mean_age = df['Age'].mean()
df['Age'] = df['Age'].fillna(mean_age)
print(df)
```

Output after filling:

Name	Age	City
Alice	23.00	New York
Bob	28.67	London
None	28.00	Paris
David	35.00	None
Emma	28.67	Sydney

Removing Incomplete Records with `dropna()`

Rows where **Name** or **City** is missing are dropped — incomplete records can compromise analysis quality.

```
df = df.dropna(subset=['Name', 'City'])
```

Before dropna()

Name	Age	City
Alice	23.00	New York
Bob	28.67	London
None	28.00	Paris
David	35.00	None
Emma	28.67	Sydney

After dropna()

Name	Age	City
Alice	23.00	New York
Bob	28.67	London
Emma	28.67	Sydney

Rows 2 (None name) and 3 (None city) are removed, leaving 3 clean records.

Data Type Conversion

The Problem

After filling missing values, the Age column is stored as `float` (e.g., 28.67). For analysis and storage efficiency, we need it as `int`.

The Benefits

Consistency

Uniform types across the column.

Storage

Integers use less memory than floats.

Analysis

Easier to work with in downstream tasks.

Converting Age to Integer with `astype()`

The `astype(int)` method casts the Age column from float to integer, truncating any decimal portion.

```
df['Age'] = df['Age'].astype(int)
print(df)
```

Output after type conversion:

Index	Name	Age	City
0	Alice	23	New York
1	Bob	28	London
4	Emma	28	Sydney

- Age values are now clean integers — no more decimal points.

Introduction to Normalization

Normalization scales numerical data into a specific range — typically 0 to 1 — so that no single feature dominates due to its magnitude.

Removes Scale Differences

Features with large ranges no longer overshadow smaller ones.

Improves ML Performance

Many algorithms converge faster and more accurately on normalized data.

Easier Comparisons

Standardized values make cross-feature analysis straightforward.

Min-Max Scaling Formula

Min-Max Scaling is the most common normalization technique. It maps each value to a position between the minimum and maximum of the column.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

X
The current value being normalized.

Xmin
The minimum value in the column.

Xmax
The maximum value in the column.

Xnorm
The resulting normalized value, always between 0 and 1.

Example Calculation: Min-Max Scaling

Our cleaned dataset has three age values: 23, 28, 28. Therefore: Minimum Age = 23, Maximum Age = 28.

Alice (Age = 23)

$$\frac{23-23}{28-23} = \frac{0}{5} = 0.0$$

Bob (Age = 28)

$$\frac{28-23}{28-23} = \frac{5}{5} = 1.0$$

Emma (Age = 28)

$$\frac{28-23}{28-23} = \frac{5}{5} = 1.0$$

Alice, as the youngest, maps to 0. Bob and Emma, at the maximum age, both map to 1.

Implementing Min-Max Scaling in Pandas

We apply the formula directly using Pandas column operations to create a new `Normalized_Age` column.

```
df['Normalized_Age'] = (
    (df['Age'] - df['Age'].min())
    /
    (df['Age'].max() - df['Age'].min())
)
```

Final DataFrame Output:

Name	Age	City	Normalized_Age
Alice	23	New York	0.0
Bob	28	London	1.0
Emma	28	Sydney	1.0

All normalized values lie between 0 ≤ Normalized_Age ≤ 1 ✓

Complete Program

The full end-to-end preprocessing pipeline in a single script:

```
import pandas as pd
import numpy as np

data = {
 'Name': ['Alice', 'Bob', None, 'David', 'Emma'],
 'Age': [23, np.nan, 28, 35, np.nan],
 'City': ['New York', 'London', 'Paris', None, 'Sydney']
}
df = pd.DataFrame(data)
print("Original DataFrame")
print(df)

# Step 1: Fill missing Age with mean
mean_age = df['Age'].mean()
df['Age'] = df['Age'].fillna(mean_age)

# Step 2: Drop rows missing Name or City
df = df.dropna(subset=['Name', 'City'])

# Step 3: Convert Age to integer
df['Age'] = df['Age'].astype(int)

# Step 4: Normalize Age using Min-Max Scaling
df['Normalized_Age'] = (
 (df['Age'] - df['Age'].min())
 / (df['Age'].max() - df['Age'].min())
)
print(df)
```

Benefits & Real-World Applications

Benefits of Data Cleaning

- ✓ Accurate analysis
- ✓ Reduced errors
- ✓ Better decision making
- ✓ Consistent datasets
- ✓ Improved ML performance

Real-World Applications

Data Science

Customer analytics, sales prediction, fraud detection.

Healthcare

Patient data cleaning, disease prediction models.

Finance

Credit risk analysis, loan approval systems.

Practice Exercise

Apply everything you've learned. Create the following DataFrame and perform the full preprocessing pipeline:

Student	Marks	City
A	70	Chennai
B	NaN	Mumbai
None	80	Delhi
D	90	None
E	NaN	Hyderabad

01

Fill Missing Marks

Use the mean of existing marks to impute `NaN` values.

02

Remove Incomplete Rows

Drop rows where Student or City is missing.

03

Convert to Integer

Cast the Marks column from float to `int`.

04

Normalize Marks

Apply Min-Max Scaling to bring all values into the 0–1 range.

Summary

- **Introduction to Data Preprocessing**
  - Importance of data cleaning and preprocessing
  - Overview of a preprocessing pipeline
- **Creating Datasets with Missing Values**
  - Building DataFrames containing NaN and None
  - Understanding incomplete datasets
- **Understanding and Identifying Missing Values**
  - Types of missing values
  - Detecting missing data using isnull()
  - Impact of missing values on analysis
- **Handling Missing Values**
  - Mean imputation using fillna()
  - Strategies for handling numerical and categorical missing data
  - Removing incomplete records using dropna()
- **Data Type Conversion**
  - Converting columns to appropriate data types using astype()
  - Benefits of data type consistency
- **Data Normalization**
  - Introduction to normalization
  - Min-Max Scaling technique
  - Implementing normalization in Pandas
- **Complete Data Preprocessing Pipeline**
  - Missing value imputation
  - Record removal
  - Type conversion
  - Feature normalization
- **Applications of Data Cleaning and Preprocessing**
  - Data science
  - Healthcare analytics
  - Financial analytics
- Machine learning preparation